

CSE3023

WEB-BASED APPLICATION DEVELOPMENT

Javabeans

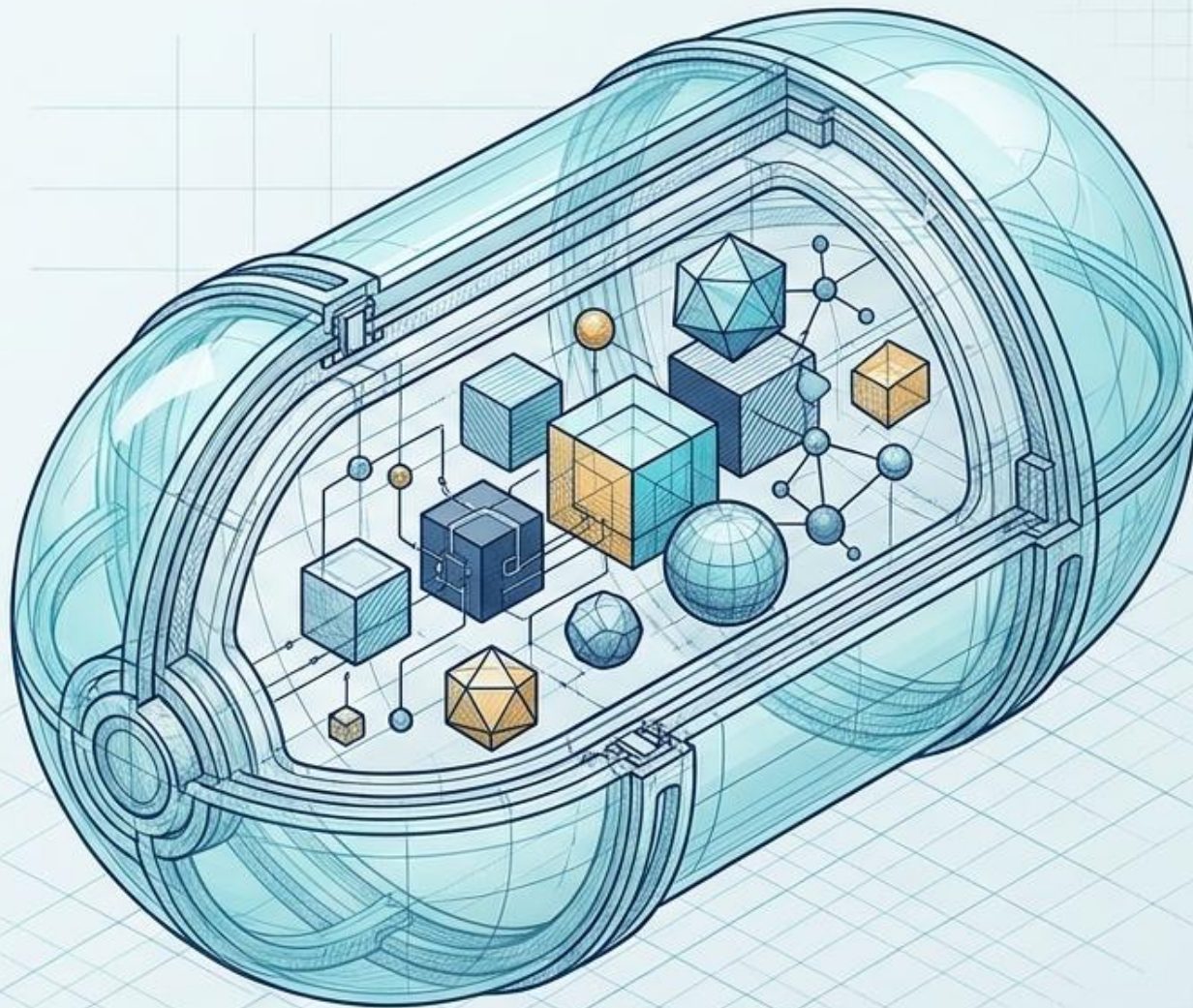
 Assoc. Prof. Ts. Dr. Wan Nural Jawahir Hj Wan Yussof
| Faculty of Computer Science & Mathematics



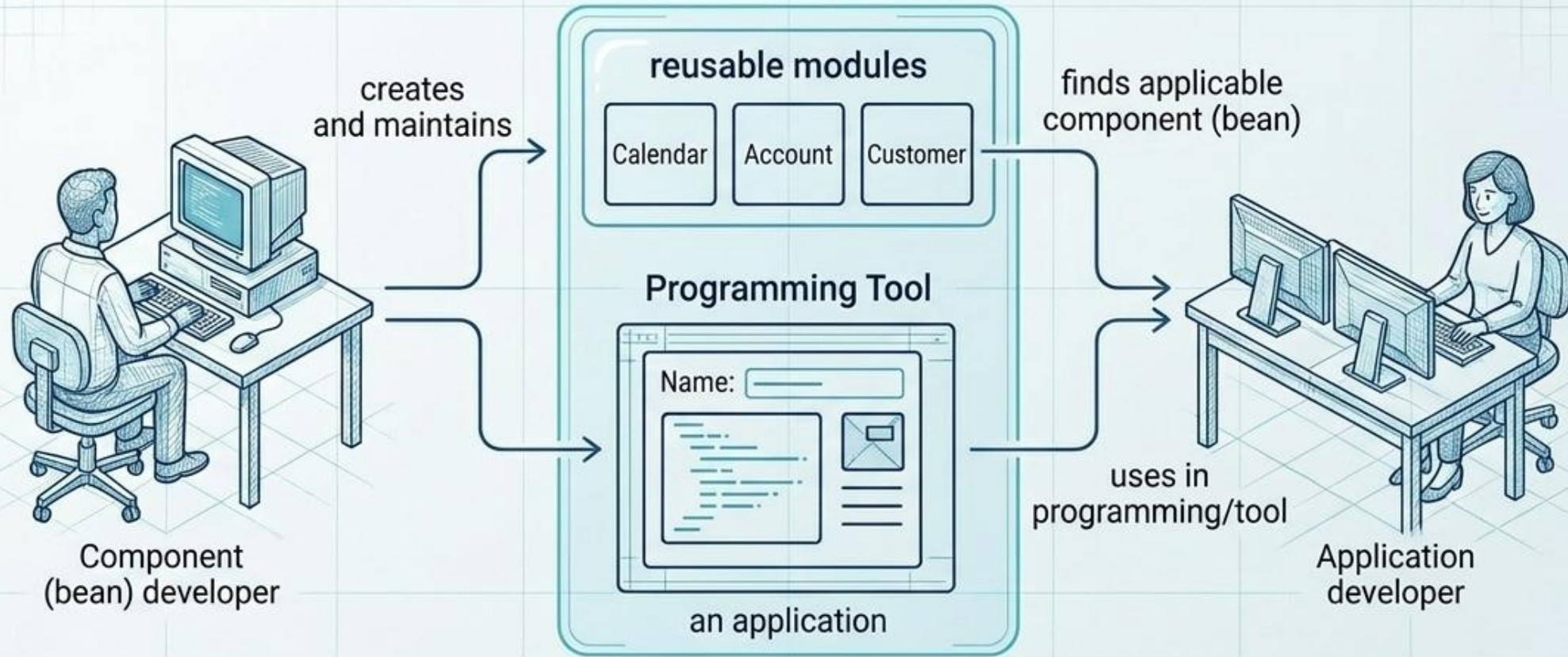
The Universal Software Capsule

A JavaBean is a portable, platform-independent model written in Java. Its primary function is to encapsulate multiple objects into a single, cohesive object.

Encapsulation: Bundles data and behaviour into a single, manageable component.



Write Once, Implement Everywhere



Reusability: Deploy across applets, servlets, and desktop applications.

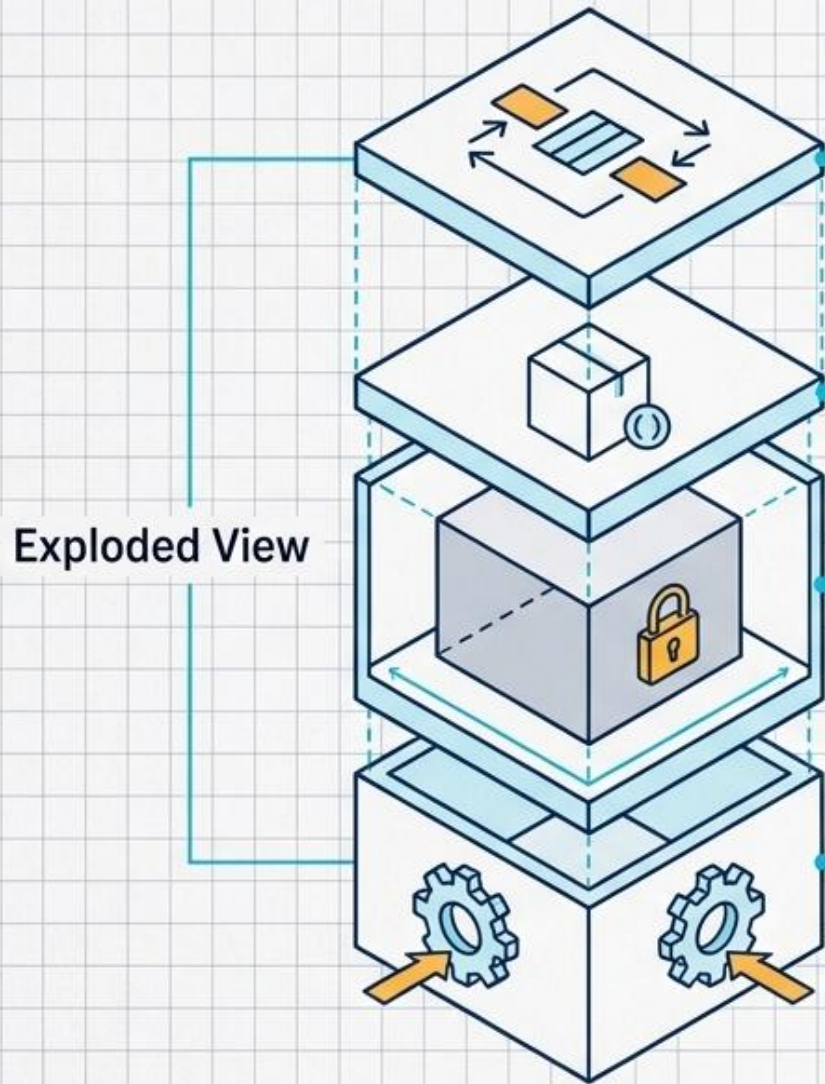


Dynamic: Highly customisable with properties that can be altered at runtime.



Scalability: Built for seamless deployment in distributed or network-based systems.

The Anatomy of a Standardised Bean



1. Implements Serializable:

Must implement `java.io.Serializable` to allow saving and restoring the bean's state.



2. Default Constructor:

Must possess a public constructor with absolutely no arguments.

```
public Bean() {  
}
```

3. Private Properties:

Internal variables must be hidden from direct external access.

```
private String data;
```

4. Public Interfaces:

Must provide specific Getter and Setter methods to read and update data.

```
public String getData() {  
    return data;  
}  
  
public void setData(String data) {  
    this.data = data;  
}
```


Accessors vs. Mutators

Getter Method (Accessor)

Visibility

Must be public

Return Type

Returns a specific value
(Cannot be void)

Naming Prefix

Begins with "get"

Arguments

Takes NO arguments

Setter Method (Mutator)

Visibility

Must be public

Return Type

Returns absolutely nothing
(Must be void)

Naming Prefix

Begins with "set"

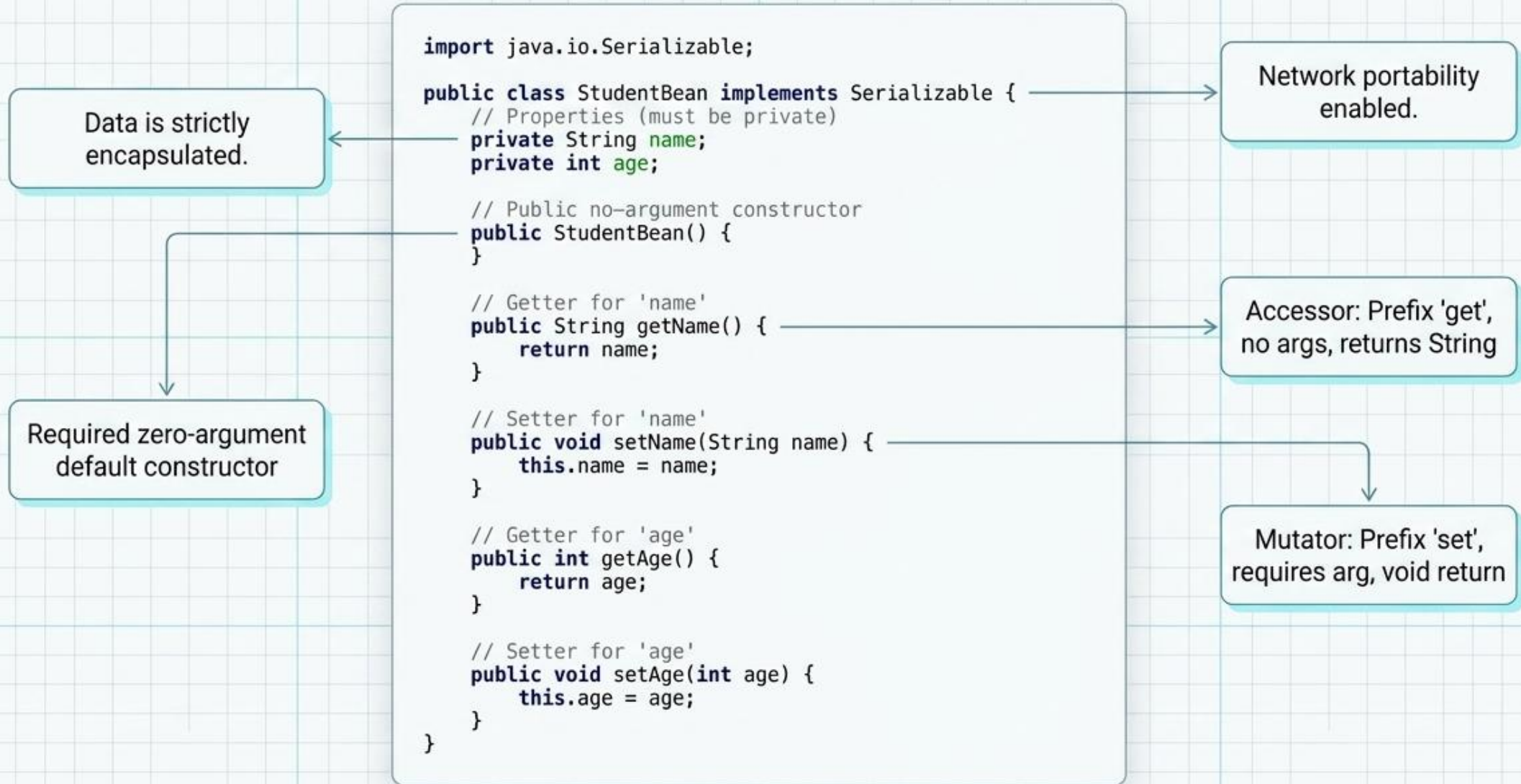
Arguments

Requires EXACTLY ONE
argument

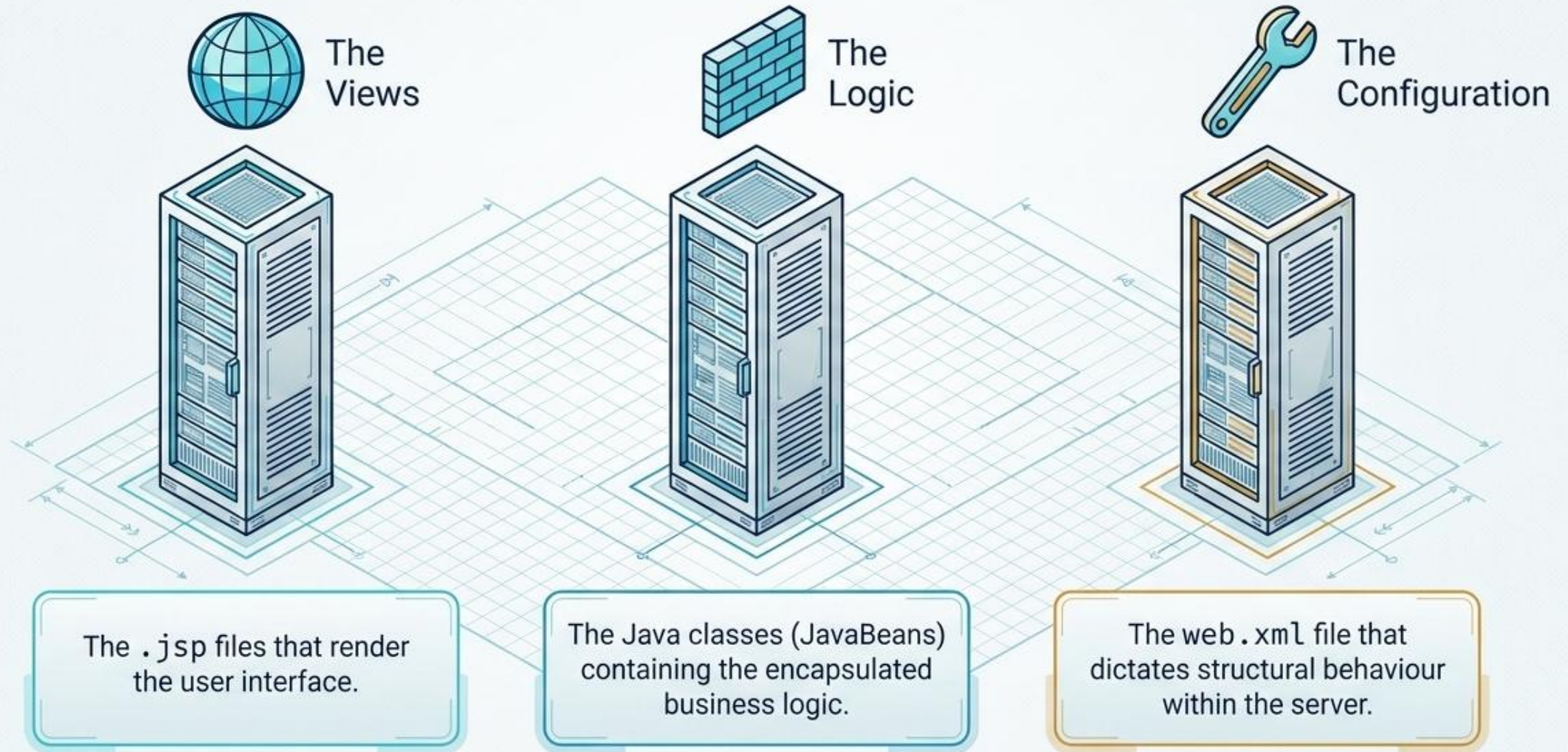


Getter = get + no arguments + returns something + public

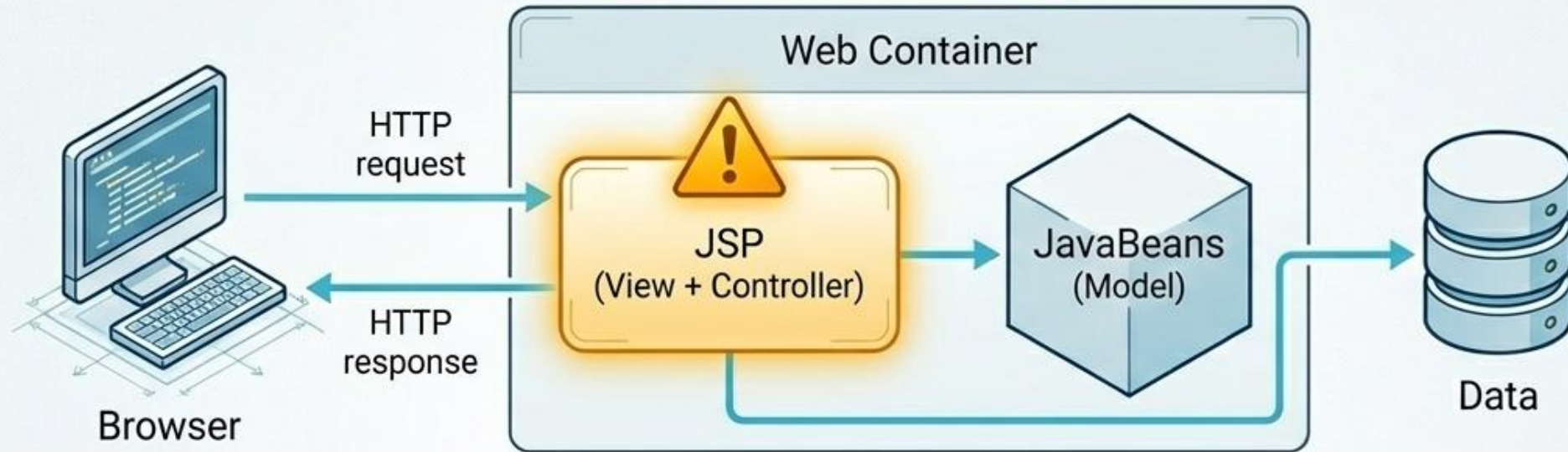
The Architect's Code Map



The Web Project Triad



JSP Model 1: The Page-Centric Bottleneck



Browser sends HTTP requests directly to the JSP.

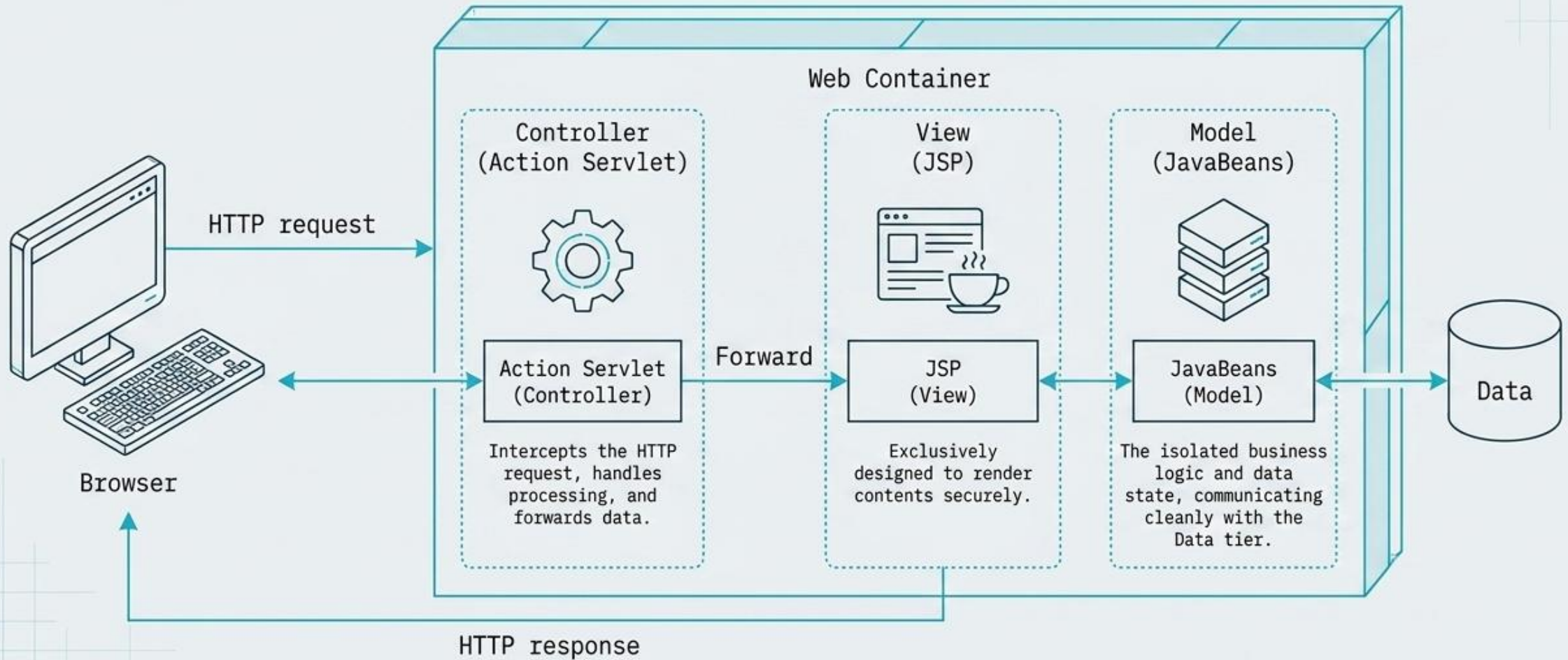
JSP acts as both View and Controller, processing the request entirely by itself.

JSP directly accesses the JavaBeans (Model) to execute business logic.



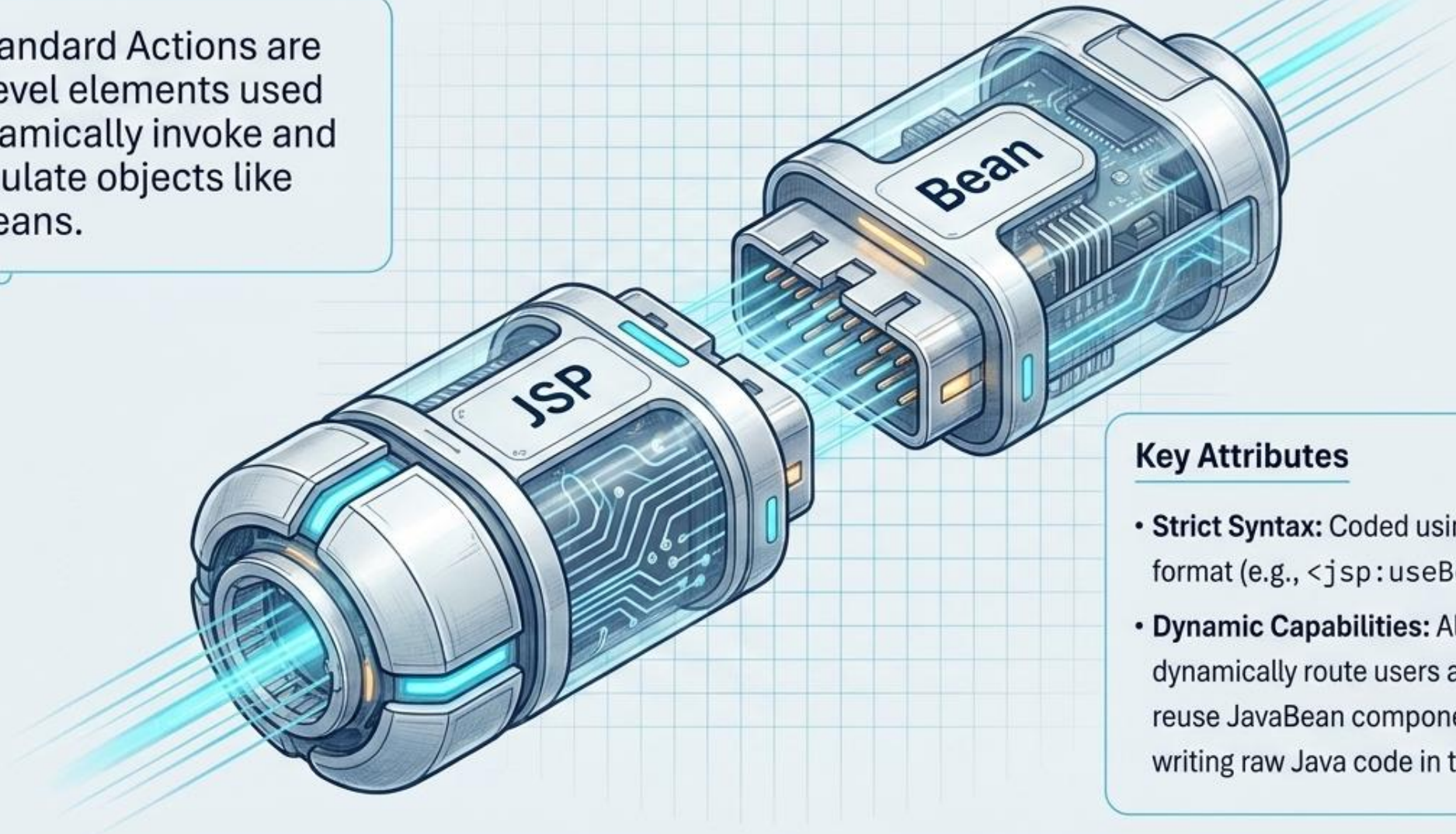
Flaw: Page-centric design forces the JSP to handle both presentation and processing, leading to tangled, hard-to-maintain code.

JSP Model 2: The MVC Framework



Standard Actions: The Architectural Connectors

JSP Standard Actions are high-level elements used to dynamically invoke and manipulate objects like JavaBeans.

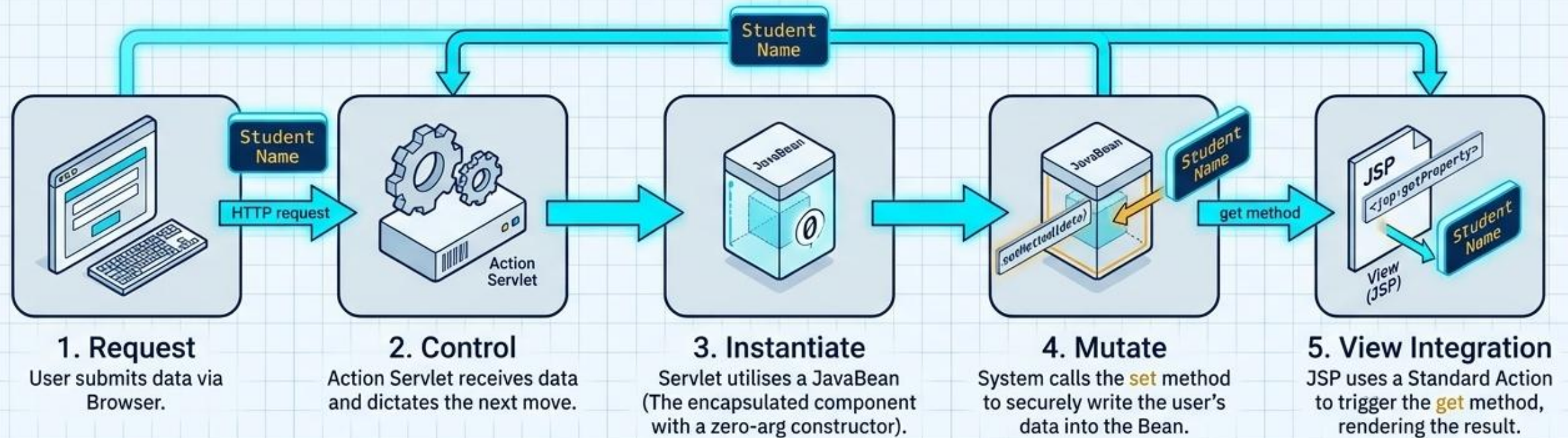


Key Attributes

- **Strict Syntax:** Coded using rigorous XML format (e.g., `<jsp:useBean>`).
- **Dynamic Capabilities:** Allows the JSP to dynamically route users and seamlessly reuse JavaBean components without writing raw Java code in the view.

The Cohesive Ecosystem

System Flow



From isolated syntax rules to a **globally scalable** architecture—the **JavaBean** standardises **data transfer** across the entire Java ecosystem.

The Developer's Toolkit: JSP Action Tags



The Core Connectors

```
<jsp:useBean id="myBean"  
  class="com.Bean.class"  
  scope="session" />
```

```
<jsp:setProperty name="myBean"  
  property="field1"  
  value="123" />
```

```
<jsp:getProperty name="myBean"  
  property="field1" />
```

Declare (The Foundation):
Declares or instantiates the
JavaBean and associates it
with a variable name.

Write (The Input):
Sets values to the
properties of the
JavaBean. It must refer
to the instantiated bean.

Read (The Output):
Retrieves values from the
JavaBean for display.



The Translation Matrix: Behind the Scenes

Declarative (JSP Action Tag)

```
java
<jsp:useBean id="myBean" class="PersonBean" scope="request" />
```

Copy

Name of object

Class name

Bean scope

Is cleanly
equivalent to

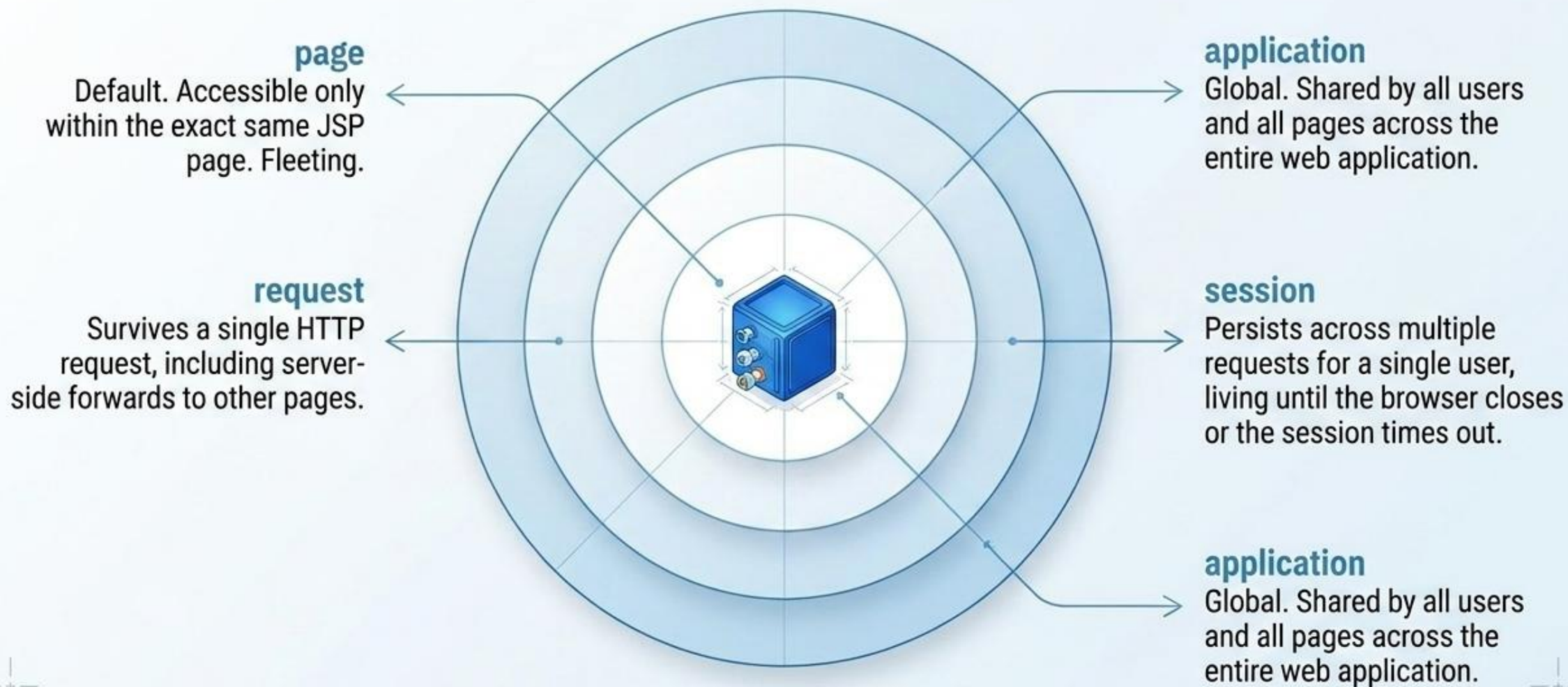
Imperative (Raw Java Scriptlet Equivalent)

```
java
PersonBean myBean = (PersonBean) request.getAttribute("myBean");
if (myBean == null) {
    myBean = new PersonBean();
    request.setAttribute("myBean", myBean);
}
```

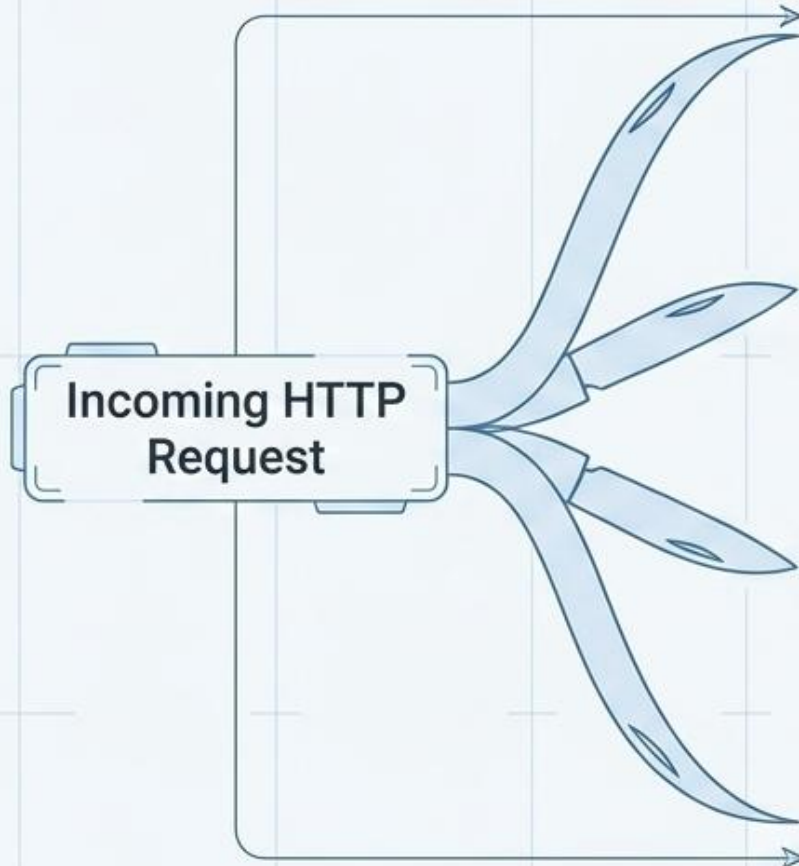
Copy

Takeaway: The standard action silently handles the heavy lifting: checking if the object exists in the current scope, instantiating it if it doesn't, and storing it for future use.

Environment & Lifespan: The Scope of a Bean



The Multi-Tool: `<jsp:setProperty>` Variations



The Blanket

```
property="*"
```

Automatically maps all incoming request parameters to matching bean properties.

The Specific

```
property="propertyName"
```

Sets one specific property matching an identically named request parameter.

The Alias

```
property="propertyName" param="parameterName"
```

Bridges a mismatch, routing a differently named parameter into the property.

The Hardcode

```
property="propertyName" value="propertyValue"
```

Bypasses the request entirely, setting the property to a fixed, direct value.

```
<% employee.setFirstName("Victor"); %>
```


Assembly Example: Forging the Raw Component

```
package model;

import java.io.Serializable;

public class Message implements Serializable {
    private String msg;

    public Message() {
    }

    public String getMsg() {
        return msg;
    }

    public void setMsg(String msg) {
        this.msg = msg;
    }
}
```

Allows state to be saved/transmitted.

Encapsulated instance variable.

Mandatory no-argument constructor.

Standardized Getter and Setter methods.

Takeaway: A JavaBean is just a standard Java class that strictly follows a specific structural contract.

Assembly Example: Wiring the Presentation

1 Step 1 (Import):
Bring the blueprint into the workspace.

```
<%@ page import="model.Message" %>
```

2 Step 2 (Declare):
Instantiate the component and assign its ID.

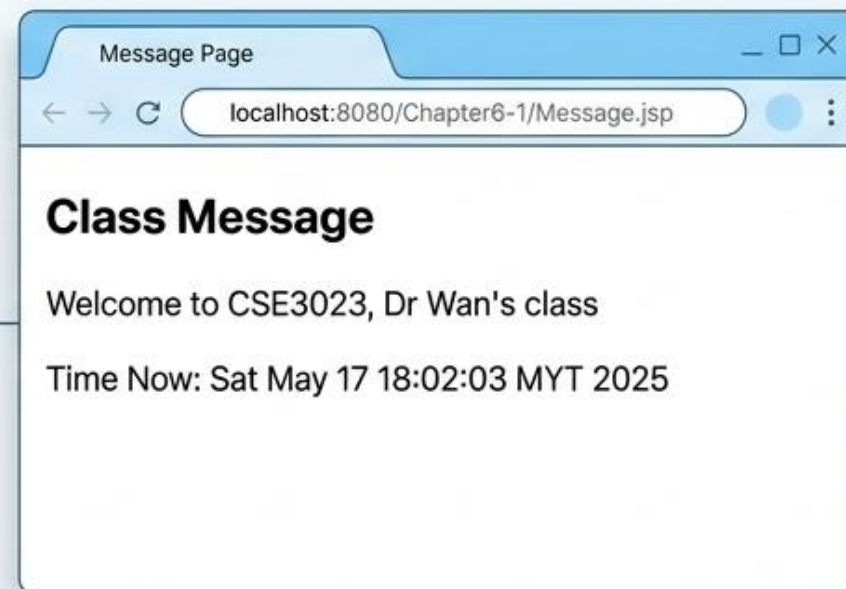
```
<jsp:useBean id="messageBean"  
  class="model.Message" scope="request" />
```

3 Step 3 (Assign):
Write the fixed value into the component.

```
<jsp:setProperty name="messageBean"  
  property="msg"  
  value="Welcome to CSE3023, Dr Wan's class" />
```

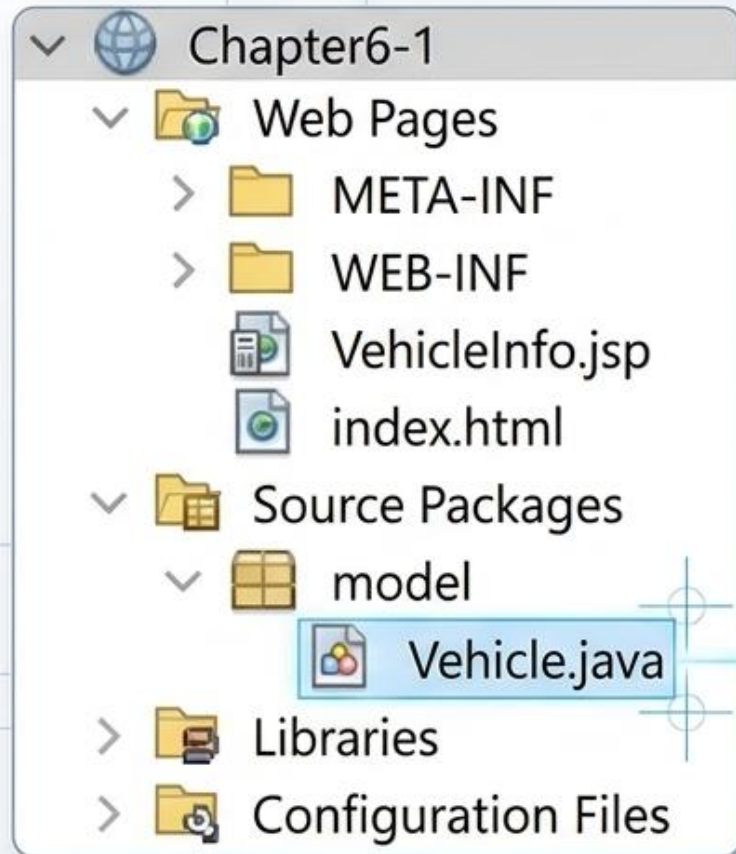
4 Step 4 (Retrieve):
Pull the stored value out and render it into the HTML structure.

```
<p><jsp:getProperty name="messageBean"  
  property="msg" /></p>
```



Scaling Up: Building a Domain Model

Real-world applications require components that hold complex states.



```
Vehicle.java ×  
  
public class Vehicle implements Serializable {  
    private String owner;  
    private String plateNo;  
  
    // Getters and Setters omitted for brevity  
}
```

The Component:
Vehicle.java
(The Data Storage)

It maintains pure Java independence, completely unaware of how it will be completed displayed in the browser. It securely holds 'owner' and 'plateNo' via standard getters and setters.

```
package model;
import java.io.Serializable;
public class Vehicle implements Serializable{
    private String owner;
    private String plateNo;

    // No-argument constructor
    public Vehicle() {
    }

    // Getters and Setters
    public String getOwner() {
        return owner;
    }

    public void setOwner(String owner) {
        this.owner = owner;
    }

    public String getPlateNo() {
        return plateNo;
    }

    public void setPlateNo(String plateNo) {
        this.plateNo = plateNo;
    }
}
```

Instance variables

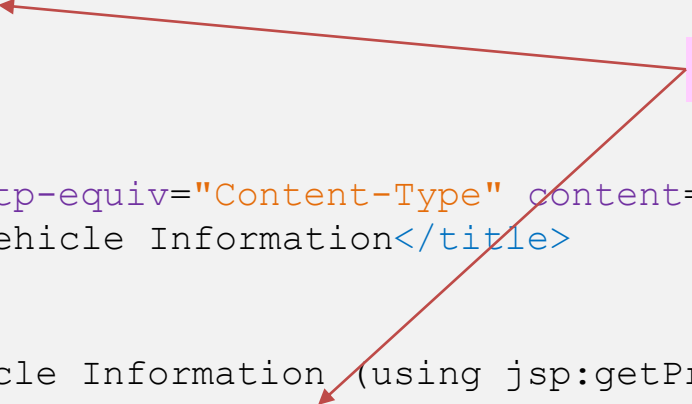
Default Constructor

Getter & Setter
methods


```
<%@page contentType="text/html" pageEncoding="UTF-8"%>
<%@page import="model.Vehicle" %>
<%@page info="Sample Page to Call JavaBeans" %>
<!-- Assign values to the JavaBean properties -->
<jsp:useBean id="myvehicle" class="model.Vehicle" scope="request" />
<jsp:setProperty name="myvehicle" property="owner" value="Ali Bin Abu" />
<jsp:setProperty name="myvehicle" property="plateNo" value="TBD1234" />

<!DOCTYPE html>
<html>
  <head>
    <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
    <title>Vehicle Information</title>
  </head>
  <body>
    <h2>Vehicle Information (using jsp:getProperty)</h2>
    <p>Owner: <jsp:getProperty name="myvehicle" property="owner" /></p>
    <p>Plate No: <jsp:getProperty name="myvehicle" property="plateNo" /></p>

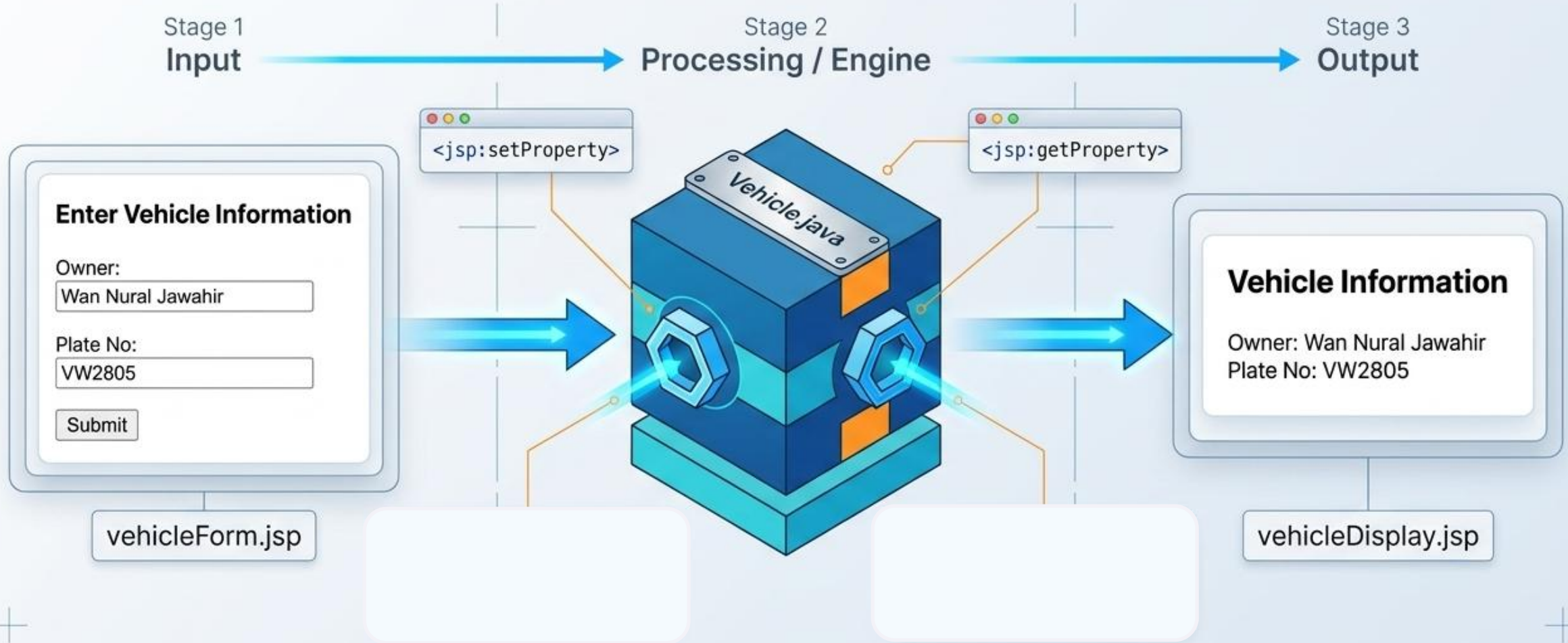
  </body>
</html>
```



A pink rectangular box with the text "Using JSP Standard Action" is positioned to the right of the code. Two red arrows originate from this box. One arrow points to the `<jsp:setProperty>` tag on line 7, and the other points to the `<jsp:getProperty>` tag on line 15.

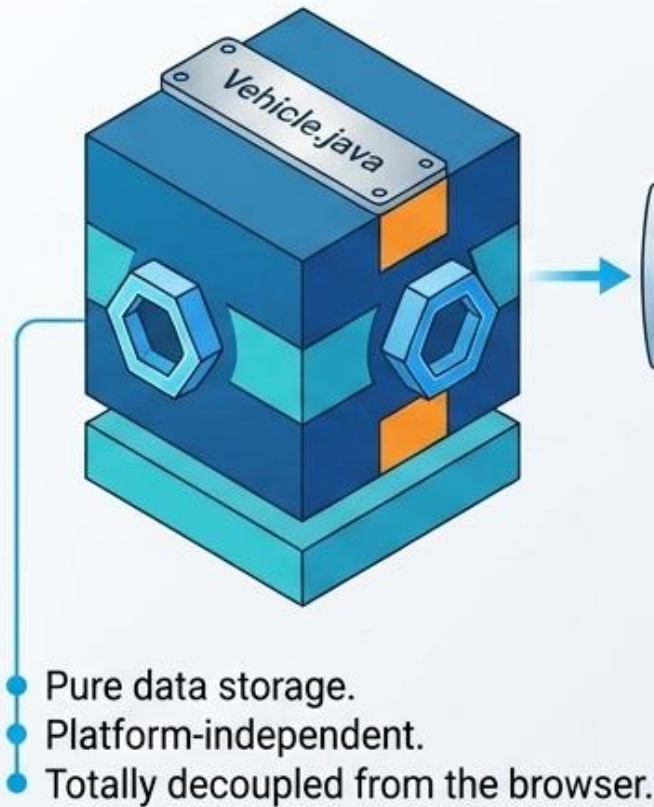
The Data Flow Engine in Action

User inputs travel from **vehicleForm.jsp**, are automatically routed into the **Vehicle.java** engine via **setProperty**, and are finally extracted by **vehicleDisplay.jsp** via **getProperty**.

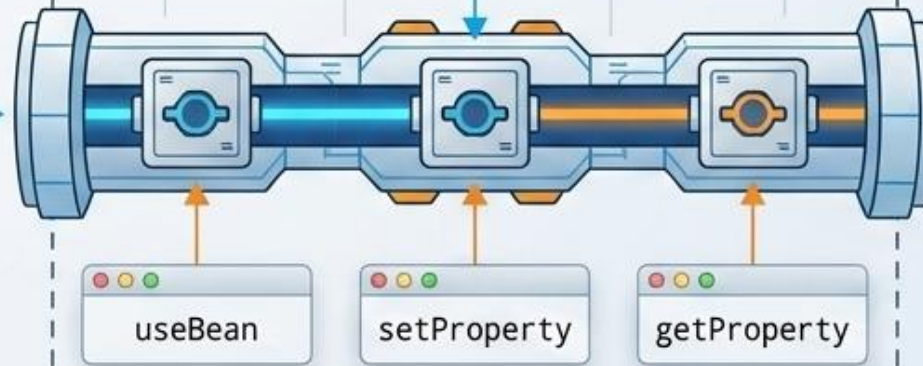


The Master Plan: MVC Architecture

Zone A: The Model (M)

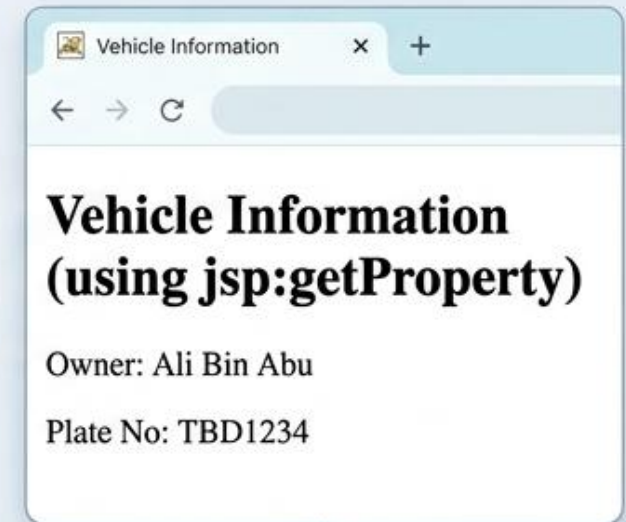


The Controller Interface



Standard actions allow us to achieve this separation of concerns cleanly, without mixing Java logic into our HTML.

Zone B: The View (V)



- The JSP Web Page (VehicleInfo.jsp).
- Pure presentation layer.
- Tailored for rendering HTML.

Why We Build with JavaBeans

Platform Independence

Keeps Java logic pure, yet integrates flawlessly into native presentation models.

Total Reusability

Software components are written once and can be reused across multiple JSPs or Servlets.

Component Assembly

Complex, enterprise-grade applications can be rapidly built by simply assembling existing Beans.

Universal Compatibility

Built on a strict API specification, guaranteeing the components will run in any Java-supported environment.

